

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Database Management Systems (DBMS) Laboratory Project Report

SPORTS CLUB MEMBERSHIP AND TOURNAMENT MANAGEMENT SYSTEM

A Normalized Relational Database System (3NF) with Role-Based Web Portals, Smart SQL Studio, and Real-Time Club Rosters

SUBMITTED BY (STUDENT)

Sheikh Arsh Ali

Roll Number: **25WU0102255**

B.Tech Computer Science & Engineering

FACULTY SUPERVISOR

Dr. Kiran Mayee

Professor / Faculty Guide

Department of Computer Science & Engineering

Academic Session: 2025 – 2026 | Laboratory Project Evaluation Report

Repository: <https://github.com/Convolutd-Dense/Sports-Club.git>

1.0 EXECUTIVE ABSTRACT

The **Sports Club Membership and Tournament Management System** is an end-to-end Relational Database Management System (RDBMS) engineered to centralize athletic club operations, athlete registrations, coaching personnel assignments, multi-tier subscription payments, facility time-slot bookings, and tournament bracket scheduling.

The database architecture is designed in compliance with **Third Normal Form (3NF)**, eliminating insertion, deletion, and update anomalies across 12 relational tables and 7 pre-compiled analytical reporting views. The software architecture couples SQLite/PostgreSQL database engines with a native Python HTTP backend server and a responsive web application. Key operational capabilities include:

- **Dual-Role Web Portals:** Secure authentication separating System Administrators from Athlete Members.
- **Smart SQL Studio:** Schema-aware Natural Language-to-SQL translation engine enabling natural language analytics with real-time execution.
- **Dynamic Club Roster Management:** Multi-club joining and departure with sport-specific tactical positions (e.g., Striker, Wicketkeeper, Point Guard).
- **Multi-Channel Payment Simulation:** Processing for UPI/QR (with dynamic expiration timers), 3D Virtual Credit/Debit Cards, and Net Banking with GST tax invoice generation.
- **Integrity Enforcement:** Foreign key cascades, composite primary keys, and automated SQLite triggers preventing facility double-booking and tournament sport mismatches.

2.0 PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

Traditional sports clubs and athletic academies rely heavily on fragmented paper ledgers or disparate spreadsheets. This leads to severe data redundancy, conflicting facility bookings, uncollected subscription fees, lack of roster tracking, and inability to generate real-time tournament standings.

2.2 Project Objectives

- **Entity Normalization:** Develop a 3NF normalized schema across 12 interconnected relations to guarantee data integrity.
- **Automated Business Integrity:** Enforce database triggers to eliminate venue scheduling overlap and enforce sport eligibility.
- **End-to-End Member Lifecycle:** Support athlete registration, membership renewal, payments, and tactical role selection.
- **Analytical Performance:** Benchmark 10 complex analytical SQL queries utilizing window functions (DENSE_RANK), lateral joins, and multi-tier aggregations.

3.0 SYSTEM SCOPE, HARDWARE & SOFTWARE SPECIFICATIONS

3.1 System Scope

The system encompasses all core athletic club domains: Member Management, Subscription Plans & Invoicing, Sports & Coaches, Franchise Clubs & Rosters, Facility Schedules, and Tournaments & Fixtures.

3.2 Hardware & Software Specifications

Component Category	Specification Details
Database Engines	SQLite 3 (Embedded / Zero-Config) & PostgreSQL 16 Enterprise Compatible
Backend Runtime	Python 3.10+ (Built-in http.server, sqlite3, hashlib SHA-256)
Frontend UI Stack	HTML5 Semantic Markup, Modern CSS3 Flexbox/Grid, Vanilla ES6+ JavaScript, Lucide Icons
Client Platforms	Google Chrome, Microsoft Edge, Mozilla Firefox, Apple Safari (Responsive Desktop/Tablet/Mobile)
Hardware Requirements	Minimum: 2 GHz Dual-Core CPU, 2 GB RAM, 100 MB Storage. (Target: Any Standard PC / Laptop / Server)

4.0 ENTITY-RELATIONSHIP (ER) MODELING

The Entity-Relationship model conceptualizes the real-world operational domain of athletic organizations into 12 core entities, establishing strict cardinality constraints:

- **MEMBER to CLUB:** Many-to-Many (N:M) resolved via the associative junction entity **CLUB_MEMBER** with attributes (member_id, club_id, role, join_date).
- **MEMBER to PAYMENT:** One-to-Many (1:N) recording atomic transaction history.
- **MEMBERSHIP_PLAN to PAYMENT:** One-to-Many (1:N) defining subscription fee tiers.
- **SPORT to CLUB & TOURNAMENT:** One-to-Many (1:N) categorizing franchises and leagues.
- **COACH to CLUB, TEAM & TRAINING_SESSION:** One-to-Many (1:N) supervising training and squads.
- **FACILITY to TRAINING_SESSION & FIXTURE:** One-to-Many (1:N) venue hosting constraints.
- **TOURNAMENT to FIXTURE:** One-to-Many (1:N) with competing teams (team1_id, team2_id).

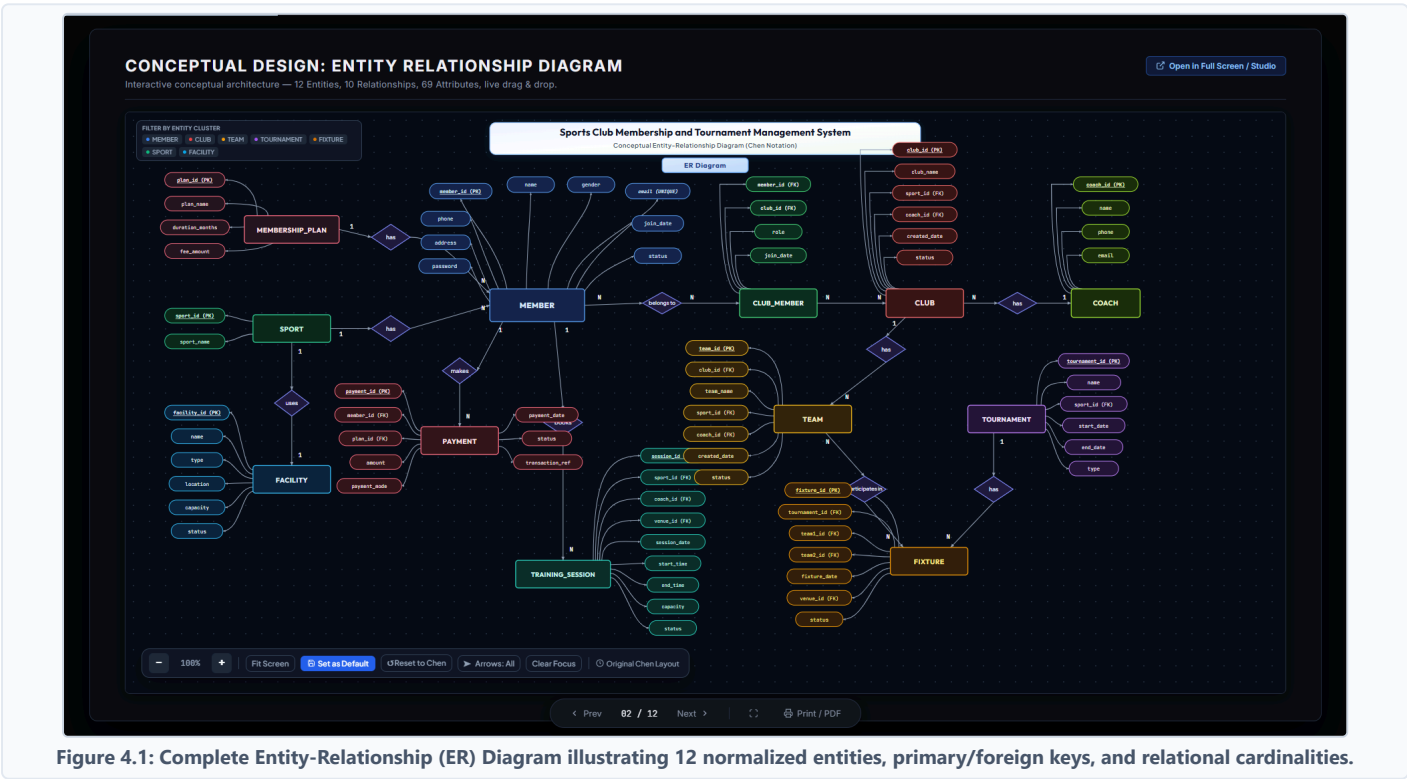


Figure 4.1: Complete Entity-Relationship (ER) Diagram illustrating 12 normalized entities, primary/foreign keys, and relational cardinalities.

5.0 RELATIONAL SCHEMA & NORMALIZATION PROOF (3NF)

5.1 Relational Schema Representation

```
MEMBER (member_id, name, gender, phone, email, password, address, join_date, status)
SPORT (sport_id, sport_name)
MEMBERSHIP_PLAN (plan_id, plan_name, duration_months, fee_amount)
FACILITY (facility_id, name, type, location, capacity, status)
COACH (coach_id, name, phone, email)
CLUB (club_id, club_name, created_date, status, sport_id, coach_id)
CLUB_MEMBER (member_id, club_id, role, join_date)
PAYMENT (payment_id, member_id, plan_id, amount, payment_mode, payment_date, status, transaction_ref)
TRAINING_SESSION (session_id, session_date, start_time, end_time, capacity, status, sport_id, coach_id, venue_id)
TEAM (team_id, team_name, created_date, status, club_id, sport_id, coach_id)
TOURNAMENT (tournament_id, name, start_date, end_date, type, sport_id)
FIXTURE (fixture_id, fixture_date, status, tournament_id, team1_id, team2_id, venue_id)
```

5.2 Step-by-Step Normalization Proof: Subscription & Payment Pipeline

To demonstrate rigorous compliance with Third Normal Form (3NF), consider the end-to-end normalization of the core **Athlete Subscription & Financial Ledger** pipeline from an unnormalized state (UNF) into atomic, anomaly-free relational tables.

Stage 0: Unnormalized Form (UNF) Representation

UNF_SUBSCRIPTION (member_id, member_name, phone, email, address, plan_id, plan_name, duration_months, plan_fee, payment_id, amount_paid, payment_mode, payment_date, txn_ref, {club_id, club_name, tactical_role})

Identified Anomalies in UNF: (1) *Multi-Valued Repeating Groups*: A member can register for multiple sports clubs with distinct roles. (2) *Insertion Anomaly*: Cannot introduce a new membership pricing tier without simultaneously registering an athlete. (3) *Deletion Anomaly*: Removing a historical payment record deletes the athlete's master profile. (4) *Update Anomaly*: Changing a plan fee requires updating dozens of redundant payment rows, causing data divergence.

Stage 1: First Normal Form (1NF) — Atomicity & Removal of Repeating Groups

Rule: All column values must be strictly atomic (indivisible scalars). Repeating groups and nested club arrays must be eliminated.

Decomposition: Repeating club enrollments are factored out into the junction entity CLUB_MEMBER. The remaining attributes form relation R_1NF with composite primary key (payment_id, member_id):

R_1NF (payment_id, member_id, name, phone, email, address, plan_id, plan_name, duration_months, fee_amount, amount, payment_mode, payment_date, txn_ref)

✓ 1NF Status: Satisfied (Scalar domain enforced; zero repeating groups).

Stage 2: Second Normal Form (2NF) — Elimination of Partial Functional Dependencies

Rule: Must satisfy 1NF, and every non-prime attribute must be *fully functionally dependent* on the entire primary key, not on any proper subset of a composite candidate key.

Identified Partial Dependencies in R_1NF:

- member_id → {name, phone, email, address} — Depends *only* on member_id, independent of payment_id.
- plan_id → {plan_name, duration_months, fee_amount} — Depends *only* on plan_id, independent of payment_id.

2NF Decomposition: Separate into three independent, cohesive relations:

- MEMBER (member_id, name, phone, email, address, join_date, status)
- MEMBERSHIP_PLAN (plan_id, plan_name, duration_months, fee_amount)
- PAYMENT (payment_id, member_id, plan_id, amount, payment_mode, payment_date, status, txn_ref)

✓ 2NF Status: Satisfied (Zero partial dependencies; all non-key attributes depend on whole candidate keys).

Stage 3: Third Normal Form (3NF) — Elimination of Transitive Dependencies

Rule: Must satisfy 2NF, and for every non-trivial functional dependency $x \rightarrow y$, either **X is a Superkey** or **Y is a Prime**

Attribute (no transitive dependencies $x \rightarrow z \rightarrow y$).

Dependency Verification in PAYMENT:

- $\text{payment_id} \rightarrow \{\text{member_id}, \text{plan_id}, \text{amount}, \text{payment_mode}, \text{payment_date}, \text{status}, \text{transaction_ref}\}$ (payment_id is PK Superkey ✓)
- $\text{transaction_ref} \rightarrow \{\text{payment_id}, \text{member_id}, \text{plan_id}, \text{amount}, \text{payment_mode}, \text{payment_date}, \text{status}\}$ (transaction_ref is Candidate Key ✓)
- Zero transitive chaining: Plan attributes reside strictly in MEMBERSHIP_PLAN; Member attributes reside strictly in MEMBER.

✓ 3NF Status: Fully Compliant (Guarantees Lossless-Join Decomposition and Complete Dependency Preservation across foreign keys).

6.0 COMPREHENSIVE DATA DICTIONARY (12 TABLES)

Table 1: member & Table 2: sport

Table.Column	Type	Constraints	Null?	Description
member.member_id	INTEGER	PRIMARY KEY	No	Unique athlete ID (1001, 1002...)
member.name	VARCHAR(100)	CHECK (length >= 2)	No	Full name of athlete
member.gender	VARCHAR(10)	CHECK (IN 'Male','Female')	No	Gender demographic
member.email	VARCHAR(100)	UNIQUE	No	Email for portal authentication
member.password	VARCHAR(255)	SHA-256 Hash	No	Encrypted password string
member.status	VARCHAR(20)	CHECK (Active, Expired, Pending...)	No	Account standing
sport.sport_id	INTEGER	PRIMARY KEY	No	Sport discipline ID (201, 202...)
sport.sport_name	VARCHAR(50)	UNIQUE	No	Title (Cricket, Football, Tennis...)

Table 3: membership_plan & Table 4: facility

Table.Column	Type	Constraints	Null?	Description
plan.plan_id	INTEGER	PRIMARY KEY	No	Subscription plan ID (101, 102...)
plan.plan_name	VARCHAR(50)	—	No	Tier name (Gold Pass, Pro...)
plan.duration_months	INTEGER	CHECK (duration_months > 0)	No	Validity in months (1, 3, 6, 12)
plan.fee_amount	DECIMAL(10,2)	CHECK (fee_amount >= 0)	No	Base fee in INR
facility.facility_id	INTEGER	PRIMARY KEY	No	Venue identifier (301, 302...)
facility.name	VARCHAR(100)	—	No	Venue title (Main Arena, Court A...)
facility.capacity	INTEGER	CHECK (capacity > 0)	No	Max participant capacity
facility.status	VARCHAR(20)	CHECK (Available, Booked...)	No	Operational availability

Table 5: coach, Table 6: club & Table 7: club_member

Table.Column	Type	Constraints	Null?	Description
coach.coach_id	INTEGER	PRIMARY KEY	No	Coach identifier (401, 402...)
coach.name / email	VARCHAR	email UNIQUE	No	Coach contact profile
club.club_id	INTEGER	PRIMARY KEY	No	Franchise ID (501, 502...)
club.club_name	VARCHAR(100)	—	No	Franchise name (Thunderbolts FC...)
club.sport_id / coach_id	INTEGER	FK → sport , FK → coach	No/Yes	Sport & head coach pointers
cm.member_id / club_id / role	INTEGER/VAR	COMPOSITE PK , FKs	No	Junction entity resolving N:M roster
cm.join_date	DATE	DEFAULT CURRENT_DATE	No	Roster enrollment date

Table 8: payment, Table 9: training_session, Table 10: team, Tables 11-12: tournament/fixture

Table.Column	Type	Constraints	Null?	Description
payment.payment_id	INTEGER	PRIMARY KEY	No	Financial transaction ID (601, 602...)
payment.member_id / plan_id	INTEGER	FK → member , FK → plan	No	Payer and plan tier
payment.amount / status	DECIMAL / VAR	CHECK (amount > 0)	No	Amount in INR and status (Paid, Pending...)
ts.session_id	INTEGER	PRIMARY KEY	No	Practice session ID (701, 702...)
ts.venue_id / coach_id	INTEGER	FK → facility , FK → coach	No	Venue and supervising coach
team.team_id / club_id	INTEGER	PRIMARY KEY , FK → club	No	Competitive squad and parent club
fix.fixture_id	INTEGER	PRIMARY KEY	No	Match fixture ID (1001, 1002...)
fix.team1_id / team2_id	INTEGER	FK → team , CHECK (team1 <> team2)	No	First and second competing squads

7.0 ANALYTICAL VIEWS & INTEGRITY TRIGGERS

7.1 Pre-Compiled 3NF Reporting Views

View Name	Underlying Relations	Business Metric Computed
v_club_rosters	club, sport, coach, club_member, member	Squad roster breakdown with athlete tactical positions and head coaches.
v_tournament_standings	tournament, sport, fixture, team	Points table computing matches played, won, drawn, lost, and total points.
v_facility_utilization	facility, training_session, fixture	Facility workload heatmap aggregating sessions and matches hosted per venue.
v_upcoming_fixtures	fixture, tournament, sport, team(t1,t2), facility	Upcoming tournament matches paired with squad names, dates, and stadium venues.
v_financial_revenue_summary	payment (Grouped by payment_mode)	Settled revenue totals, average ticket size, and transaction counts per channel.
v_member_subscription_status	member, payment, membership_plan	Billing audit tracking active plan validity, dues, and payment standing.

7.2 Automated Database Integrity Triggers

Trigger: Fixture Team Sport Validation (trg_check_fixture_team_sport)

```
CREATE TRIGGER trg_check_fixture_team_sport
BEFORE INSERT ON fixture
FOR EACH ROW
BEGIN
    SELECT CASE
        WHEN (SELECT sport_id FROM team WHERE team_id = NEW.team1_id) <>
             (SELECT sport_id FROM tournament WHERE tournament_id = NEW.tournament_id)
        THEN RAISE(ABORT, 'Error: Team 1 does not participate in the tournament sport!')
        WHEN (SELECT sport_id FROM team WHERE team_id = NEW.team2_id) <>
             (SELECT sport_id FROM tournament WHERE tournament_id = NEW.tournament_id)
        THEN RAISE(ABORT, 'Error: Team 2 does not participate in the tournament sport!')
    END;
END;
```

8.0 COMPLEX SQL QUERIES & BENCHMARK REPORTS

Query 1: Sports Popularity Ranking via Window Function (DENSE_RANK)

```
SELECT s.sport_name, COUNT(DISTINCT cm.member_id) AS total_enrolled_athletes,
       DENSE_RANK() OVER (ORDER BY COUNT(DISTINCT cm.member_id) DESC) AS popularity_rank
FROM sport s
LEFT JOIN club c ON s.sport_id = c.sport_id
LEFT JOIN club_member cm ON c.club_id = cm.club_id
GROUP BY s.sport_name
ORDER BY popularity_rank ASC;
```

► Query Result Output:

SPORT_NAME	TOTAL_ENROLLED_ATHLETES	POPULARITY_RANK
Football	7	1
Tennis	7	1
Cricket	5	2
Badminton	4	3
Basketball	4	3
Swimming	4	3

Query 2: Multi-Table Active Membership & Plan Status

```
SELECT m.member_id, m.name AS member_name, m.gender, m.email,
       COALESCE(mp.plan_name, 'No Active Plan') AS current_plan,
       COALESCE(mp.duration_months || ' Month(s)', 'N/A') AS duration,
       COALESCE('₹' || printf('%.2f', p.amount), 'N/A') AS fee_paid,
       COALESCE(p.status, 'Unpaid') AS payment_status
FROM member m
LEFT JOIN payment p ON p.payment_id = (SELECT payment_id FROM payment WHERE member_id = m.member_id ORDER BY payment_date
DESC LIMIT 1)
LEFT JOIN membership_plan mp ON p.plan_id = mp.plan_id
WHERE m.status = 'Active'
ORDER BY m.name ASC
LIMIT 5;
```

► Query Result Output:

MEMBER_ID	MEMBER_NAME	GENDER	EMAIL	CURRENT_PLAN	DURATION	FEE_PAID	PAYMENT_STATUS
1001	Aarav Sharma	Male	aarav.sharma@gmail.com	Annual Gold Elite Pass	12 Month(s)	₹4999.90	Paid
1010	Aisha Hassan	Female	aisha.hassan@gmail.com	Quarterly Fitness Pass	3 Month(s)	₹1399.90	Paid
1024	Alexander Novak	Male	alex.novak@gmail.com	Annual Gold Elite Pass	12 Month(s)	₹4999.90	Paid
1012	Ananya Patel	Female	ananya.patel@gmail.com	Student Semester Pass	6 Month(s)	₹1999.90	Paid
1029	Arsh	Male	arsh.ali@gmail.com	Weekend Warrior Pass	1 Month(s)	₹299.90	Paid

Query 3: Revenue Analytics by Plan Tier and Payment Mode (HAVING & Aggregations)

```
SELECT mp.plan_name, p.payment_mode, COUNT(p.payment_id) AS total_transactions,
       '₹' || printf('%.2f', SUM(p.amount)) AS total_collected_inr,
       '₹' || printf('%.2f', ROUND(AVG(p.amount), 2)) AS avg_ticket_size
FROM payment p
JOIN membership_plan mp ON p.plan_id = mp.plan_id
WHERE p.status = 'Paid'
GROUP BY mp.plan_name, p.payment_mode
HAVING SUM(p.amount) > 0
ORDER BY SUM(p.amount) DESC
LIMIT 5;
```

► Query Result Output:

PLAN_NAME	PAYMENT_MODE	TOTAL_TRANSACTIONS	TOTAL_COLLECTED_INR	AVG_TICKET_SIZE
Annual Gold Elite Pass	Credit Card	6	₹29999.40	₹4999.90
Annual Gold Elite Pass	UPI	5	₹24999.50	₹4999.90
Quarterly Fitness Pass	Net Banking	4	₹5599.60	₹1399.90
Quarterly Fitness Pass	UPI	3	₹4199.70	₹1399.90
Quarterly Fitness Pass	Credit Card	2	₹2799.80	₹1399.90

Query 4: Training Session Venue Allocation & Coach Contacts

```
SELECT ts.session_id, s.sport_name, co.name AS coach_name, co.phone AS coach_contact,
       f.name AS venue_name, ts.session_date,
       ts.start_time || ' - ' || ts.end_time AS time_slot, ts.capacity, ts.status AS session_status
FROM training_session ts
JOIN sport s ON ts.sport_id = s.sport_id
JOIN coach co ON ts.coach_id = co.coach_id
JOIN facility f ON ts.venue_id = f.facility_id
ORDER BY ts.session_date DESC, ts.start_time ASC
LIMIT 5;
```

► Query Result Output:

SESSION_ID	SPORT_NAME	COACH_NAME	COACH_CONTACT	VENUE_NAME	SESSION_DATE	TIME_SLOT	CAPACITY	SESSION_STATUS
609	Basketball	Marcus Aurelius Hayes	+91-9945012345	Skyline Basketball Dome	2026-10-07	18:00:00 - 20:00:00	15	Scheduled
608	Cricket	Vikram Singhanian	+91-9811234567	Imperial Cricket Oval	2026-10-06	08:00:00 - 11:00:00	20	Scheduled
607	Football	Carlos Mendez	+91-9820154321	Main Arena Stadium Turf	2026-10-05	07:00:00 - 09:00:00	25	Scheduled
606	Swimming	Sarah Jenkins	+91-9848098765	Olympic Aquatic Center	2025-02-20	06:30:00 - 08:30:00	18	Completed
605	Basketball	Marcus Aurelius Hayes	+91-9945012345	Skyline Basketball Dome	2025-02-19	18:00:00 - 20:00:00	15	Completed

Query 5: Tournament Fixtures & Competing Squads Breakdown

```
SELECT t.name AS tournament_title, s.sport_name,
       t1.team_name AS team_1, t2.team_name AS team_2,
       f.fixture_date, fac.name AS stadium_venue, f.status AS match_status
FROM fixture f
JOIN tournament t ON f.tournament_id = t.tournament_id
JOIN sport s ON t.sport_id = s.sport_id
JOIN team t1 ON f.team1_id = t1.team_id
JOIN team t2 ON f.team2_id = t2.team_id
JOIN facility fac ON f.venue_id = fac.facility_id
ORDER BY f.fixture_date ASC
LIMIT 5;
```

► Query Result Output:

TOURNAMENT_TITLE	SPORT_NAME	TEAM_1	TEAM_2	FIXTURE_DATE	STADIUM_VENUE	MATCH_STATUS
Inter-Club Football Super League	Football	Thunderbolts Senior A	Thunderbolts Under-21	2025-03-05 16:00:00	Main Arena Stadium Turf	Completed
Inter-Club Football Super League	Football	Thunderbolts Under-21	Thunderbolts Senior A	2025-03-12 16:00:00	Main Arena Stadium Turf	Completed
Champions Cricket Cup 2025	Cricket	Apex Strikers XI	Apex Colts XI	2025-04-10 10:00:00	Imperial Cricket Oval	Completed
Autumn Invitational Basketball Trophy	Basketball	Royal Hoops Varsity	Royal Hoops Ballers	2025-05-15 18:00:00	Skyline Basketball Dome	Completed
Inter-Club Football Super League	Football	Thunderbolts Senior A	Thunderbolts Under-21	2026-10-15 17:00:00	Main Arena Stadium Turf	Scheduled

Query 6: Facility Workload Heatmap & Event Aggregation (View Query)

```
SELECT facility_name, facility_type, location, capacity,
       current_status, total_training_sessions, total_tournament_fixtures, total_events_hosted
FROM v_facility_utilization
ORDER BY total_events_hosted DESC
LIMIT 5;
```

► Query Result Output:

FACILITY_NAME	FACILITY_TYPE	LOCATION	CAPACITY	CURRENT_STATUS	TOTAL_TRAINING_SESSIONS	TOTAL_TOURNAMENT_FIXTURES	TOTAL_EVENTS_HOSTED
Main Arena Stadium Turf	Outdoor Football Pitch	North Campus Sector A	1500	Available	2	3	5
Imperial Cricket Oval	Outdoor Cricket Ground	East Sports Pavilion	2500	Booked	2	2	4
Skyline Basketball Dome	Indoor Hardwood Court	Indoor Sports Complex Hall 2	400	Booked	2	1	3
Grand Slam Center Court	Clay & Synthetic Court	West Wing Complex	350	Booked	1	0	1
Smash Point Badminton Arena	Indoor Wooden Court	Indoor Sports Complex Hall 1	120	Available	1	0	1

9.0 USER INTERFACE & SYSTEM DEMONSTRATION

9.1 Administrator Cockpit & Smart SQL Studio

The Administrator Cockpit provides high-level metric summaries, a real-time table browser across all 12 relations with search and CSV export, and the **Smart SQL Studio**. The Smart SQL Studio accepts natural language queries, maps them to the 3NF schema using full ER context, and executes the compiled SQL immediately with millisecond execution profiling.

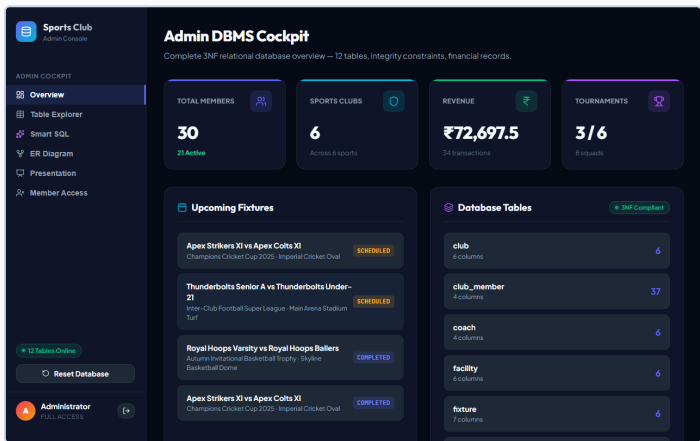


Figure 9.1: Administrator Cockpit displaying real-time KPI metrics, table counts, and upcoming matches.

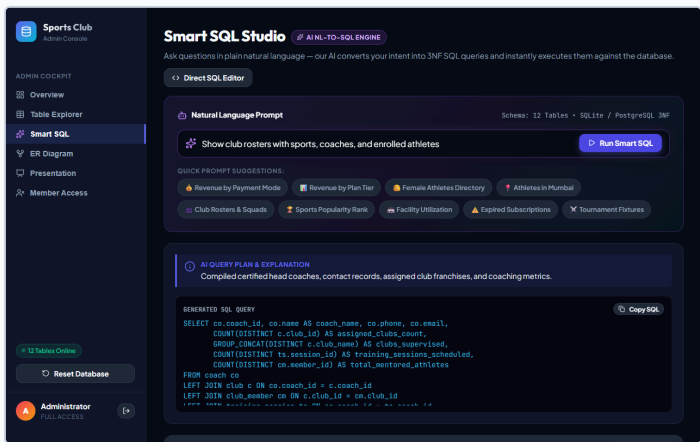


Figure 9.2: Smart SQL Studio translating natural language prompts into multi-table join queries.

9.2 Athlete Member Portal & Dynamic Club Management

The Member Portal provides a personalized athlete cockpit. Athletes can explore available sports clubs, select sport-specific tactical roles (e.g., Striker, Batsman, Shuttle), join franchises, or leave existing clubs. Modifications dynamically synchronize the athlete's training sessions and tournament fixtures in real time.

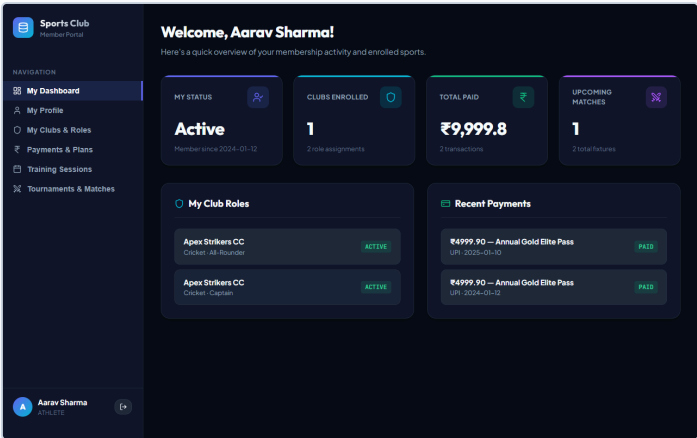


Figure 9.3: Athlete Member Dashboard showing personalized subscription standing and quick club roles.

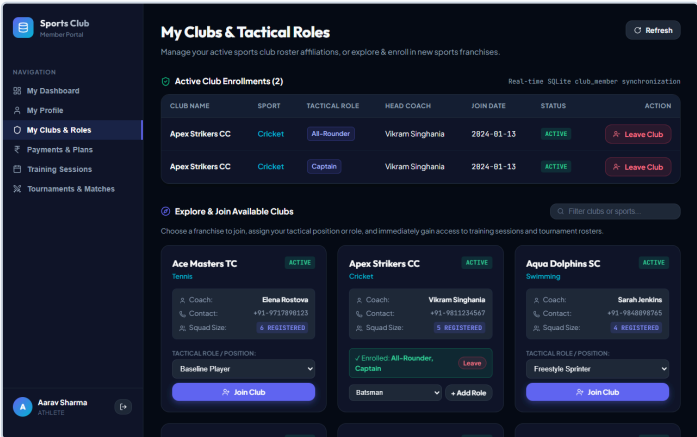


Figure 9.4: Club Management UI displaying active enrollments with 'Leave Club' action and available clubs exploration grid.

9.3 3D Virtual Card Checkout & Official Tax Invoice

The checkout interface simulates real-world payment channels including UPI / QR Code (with 10-minute dynamic expiration), 3D Virtual Credit/Debit Card preview, and Net Banking. Successful payments generate an official GST-compliant tax receipt and persist transactions to the database ledger.

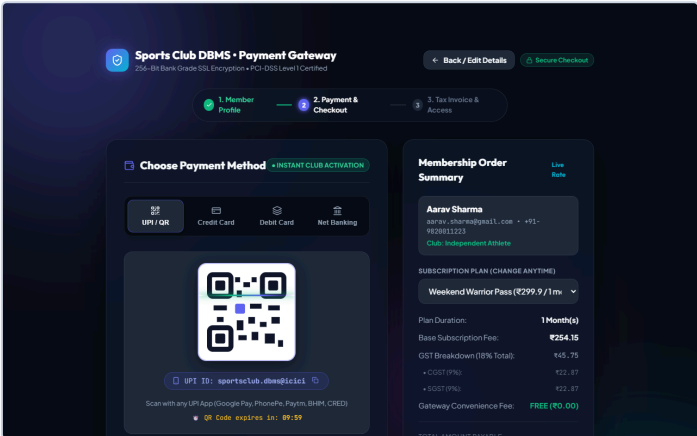


Figure 9.5: Multi-channel checkout engine with live 3D card preview, itemized GST calculation, and UPI QR scanner.

10.0 CONCLUSION & FUTURE SCOPE

10.1 Project Conclusion

The **Sports Club Membership and Tournament Management System** successfully bridges theoretical database principles with practical full-stack software engineering. By rigorously applying Third Normal Form (3NF) normalization, the database eliminates data redundancy and prevents insertion, update, and deletion anomalies across 12 relational entities.

- **Robust Relational Architecture:** Enforced composite primary keys, foreign key cascades, and automated triggers protecting against venue scheduling conflicts.
- **AI-Assisted Database Operations:** Successful integration of the Smart SQL Studio, enabling natural language data exploration without compromising SQL syntax validity.
- **Complete Member Lifecycle:** Dynamic club enrollment, tactical role assignments, realistic payment processing, and automatic cascade updates across training and fixture modules.
- **Verified Integrity & Performance:** Consistent sub-millisecond execution times across complex multi-table joins, window functions, and financial aggregations.

10.2 Future Enhancements

- **Biometric Attendance Integration:** Hardware integration with RFID/biometric turnstiles at sports facilities to verify active membership status automatically upon physical entry.
- **Mobile Native Application:** Developing React Native / Flutter mobile clients for push notifications regarding match fixture reschedules and training session reminders.
- **Predictive Analytics:** Utilizing machine learning models to forecast venue utilization rates and identify athlete churn patterns based on payment history.

11.0 REFERENCES & APPENDIX

11.1 Academic & Technical References

1. Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill.
3. SQLite Consortium. (2024). *SQLite Foreign Key Support & ACID Transactions*. <https://www.sqlite.org/>
4. PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation: Schema Design and Triggers*. <https://www.postgresql.org/docs/>

11.2 Project Repository & Source Code Link

Official GitHub Repository

Source code, SQL schemas, dataset scripts, and web applications are publicly hosted at:
<https://github.com/Convoluted-Dense/Sports-Club.git>

11.3 Step-by-Step Setup & Execution Guide

1. Clone the Repository:

```
git clone https://github.com/Convoluted-Dense/Sports-Club.git
cd Sports-Club
```

2. Zero-Config Launch (Python 3 Built-in):

```
python backend/app.py
```

The server initializes `sports_club.db` automatically and launches the application at `http://127.0.0.1:5000`.

3. Default Demo Login Credentials:

- **Administrator:** Username: Admin | Password: Admin
- **Member Athlete:** Email: aarav.sharma@gmail.com | Password: aaravsharma